

to prevent abuse. Session storage in Redis improves performance compared to database-backed sessions. Cache invalidation using Django signals ensures data freshness when models change.

For rate limiting, you can store a counter in Redis with an expiration time. Each request increments the counter. If the counter exceeds the limit, reject the request. Redis's atomic increment operation makes this thread safe.

Session storage in Redis eliminates database queries for session data on every authenticated request. Configure Django's `SESSION_ENGINE` to use the cache backend, and sessions automatically store in Redis instead of the database.

Automatic cache invalidation prevents stale data. Use Django signals to delete cache entries when models are saved or deleted. This ensures users always see current data without manual cache management.

Asynchronous task processing with Celery

Celery is a distributed task queue that offloads time-consuming operations to background workers. When users upload images, generate reports, or trigger external API calls, they shouldn't wait for completion. Celery processes these tasks asynchronously while the web request returns immediately.

Installing Celery requires the `celery` and `redis` packages. Create a Celery application in your Django project, configure it to use Redis as the message broker, and define tasks using the `shared_task` decorator. Tasks are regular Python functions that Celery can execute asynchronously.

Calling tasks from views uses the `delay` method, which sends the task to Celery workers and returns immediately. The user receives an instant response while processing happens in the background. This pattern works for emails, file processing, payment processing, data exports, and any operation that doesn't require immediate results.

Celery Beat adds scheduled task execution. Define periodic tasks in Django settings using `crontab` schedules. Common uses include refreshing materialised views, sending digest emails, cleaning up old data, and generating scheduled reports.

Flower provides a web interface for monitoring Celery tasks. It shows active tasks, completed tasks, failed tasks, and worker status. This visibility is essential for production systems where you need to track background job execution.

```
#Install Celery
pip install celery redis
#myproject/celery.py
import os
from celery import Celery
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.
settings')
```

```
app = Celery('myproject')
app.config_from_object('django.conf:settings',
namespace='CELERY')
app.autodiscover_tasks()
#settings.py
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
```

Real-time features with Django Channels

Django Channels extends Django beyond HTTP to handle WebSockets, enabling real-time features like chat, notifications, and live dashboards. While Celery handles background tasks, Channels enables bidirectional communication between servers and clients.

Channels requires the `'channels'` and `'channels-redis'` packages. Configure Django to use ASGI instead of WSGI, define WebSocket consumers, and create routing for WebSocket connections. Consumers handle WebSocket events like connection, disconnection, and incoming messages.

```
#Install Channels
pip install channels channels-redis
#settings.py
INSTALLED_APPS = ['channels', ...]
ASGI_APPLICATION = 'myproject.asgi.application'
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {'hosts': [('127.0.0.1', 6379)]},
    },
}
```

Sending real-time notifications from Django code uses channel layers. When an event occurs—order confirmation, payment success, system alert—you can send a message to the channel layer, which broadcasts to connected WebSocket clients. Users receive instant notifications without polling.

Run Channels applications with Daphne, an ASGI server. Daphne handles both HTTP and WebSocket protocols, making it suitable for production deployments of Django Channels applications.

Optimising API payload sizes

Large API responses waste bandwidth and slow down clients. Optimise payloads by creating different serialisers for list and detailed endpoints. List views show minimal fields for quick browsing, while detailed views provide complete data.

Dynamic field selection lets clients specify needed fields via query parameters. Implement this with a custom serialiser that removes fields not requested. This flexibility